Guía para estudiantes
Actividad integradora: Unidades 1 y 2

Asignatura: Programación Funcional

Título del proyecto

Simulador funcional con interfaz gráfica

Propósito de la actividad

En esta actividad deberán desarrollar una aplicación en **Python** usando **Tkinter** y **Ttk-Bootstrap**, integrando de forma obligatoria contenidos de la **Unidad 1** y la **Unidad 2**.

La meta no es solamente que la aplicación funcione. La meta es que ustedes demuestren que **comprenden**, **aplican**, **analizan** y **justifican** decisiones de programación funcional dentro de un proyecto real.

Uso de inteligencia artificial

Sí se permite usar IA.

Pueden usarla para:

- proponer ideas de interfaz,
- mejorar la parte visual,
- sugerir estructuras iniciales,
- acelerar tareas mecánicas,
- refactorizar código con supervisión.

Pero ojo: usar IA no significa delegar la comprensión.

Lo que se evaluará no es si la IA les ayudó, sino si ustedes fueron capaces de:

- entender lo que la IA propuso,
- detectar errores o malas decisiones,
- corregir o adaptar el código,
- justificar técnicamente por qué su solución aplica programación funcional.

La IA es herramienta, no sustituto del criterio técnico.

Descripción general del trabajo

Cada estudiante desarrollar? de forma **individual** una aplicaci?n con interfaz gr?fica. El dominio puede variar. Algunos ejemplos posibles son:

- simulador de inventario,
- sistema de misiones,
- gestor de tareas,
- arena de combate,
- sistema de pedidos,
- administrador de personajes, habilidades u objetos,
- u otro dominio equivalente aprobado por el docente.

Lo importante es que la solución permita mostrar con claridad dónde aparecen los conceptos funcionales estudiados.

Requisitos obligatorios de la interfaz

La aplicación debe usar obligatoriamente:

- `tkinter`
- `ttkbootstrap`

La interfaz debe estar:

- ordenada,
- legible,
- consistente,
- visualmente cuidada,
- pensada para un usuario real.

No se evalúa diseño profesional, pero sí **criterio visual, claridad y esfuerzo de presentación**.

Contenidos obligatorios de la Unidad 1

El proyecto debe aplicar **sí o sí** los siguientes contenidos:

- funciones de orden superior,
- pattern matching con `match/case`,

- modelado de datos usando al menos uno de estos recursos:
  - `NamedTuple`
  - `dataclass`
  - `Union`
- comparación o evidencia de enfoque funcional vs. imperativo en al menos una parte del proyecto.

Contenidos obligatorios de la Unidad 2

El proyecto debe aplicar **sí o sí** los siguientes contenidos:

- `lambda`,
- closure,
- inmutabilidad,
- pureza y efectos secundarios,
- uso de al menos una de estas herramientas funcionales:
  - `map`
  - `filter`
  - `reduce`

Restricción de diseño obligatoria

El proyecto debe separar claramente:

- **capa visual:** ventanas, widgets, botones, formularios, tablas y mensajes;
- **capa lógica:** reglas del sistema, transformaciones y decisiones del dominio;
- **capa de datos o estado:** estructuras principales del programa.

Regla crítica: no se permite concentrar toda la lógica dentro de callbacks de botones o eventos de interfaz.

Si mezclan interfaz, lógica y estado sin criterio, el proyecto pierde calidad técnica y conceptual.

Propuesta sugerida de proyecto

Opción recomendada: Arena funcional con interfaz

Pueden desarrollar una aplicación donde un jugador administre personajes, habilidades, objetos o acciones desde una interfaz gráfica.

La aplicación podría permitir, por ejemplo:

- crear personajes o entidades,
- ejecutar acciones,
- aplicar filtros,
- mostrar historial,
- calcular resultados acumulados,
- actualizar estados,
- mostrar reportes,
- administrar buffs, inventario o eventos.

Cómo se integran los temas

Unidad 1

- **Pattern matching** para procesar acciones o eventos como atacar, curar, comprar, equipar, vender, usar objeto o activar habilidad.
- **Funciones de orden superior** para transformar atributos, filtrar acciones válidas, acumular puntajes, generar reportes o procesar listas de objetos o enemigos.
- **Modelado de datos** con `dataclass`, `NamedTuple` o `Union` para representar jugador, enemigo, item, acción, estado, evento o recompensa.

Unidad 2

- **Lambda** para transformaciones pequeñas y locales.
- **Closure** para crear acciones configuradas, descuentos, buffs, multiplicadores, validadores o callbacks configurados.
- **Inmutabilidad** al actualizar el estado sin mutarlo directamente.
- **Pureza** en la lógica del dominio.
- **Efectos secundarios** en la interfaz, impresión, lectura de datos o guardado.
- Uso real de `map`, `filter` o `reduce`.

Reglas obligatorias del desarrollo

1. No se permite toda la lógica dentro de botones o eventos.
2. Toda acción importante debe pasar por funciones separadas.
3. Debe haber al menos 3 funciones puras justificadas.

4. Debe haber al menos 2 funciones impuras justificadas.
5. Debe existir al menos 1 closure real.
6. Debe existir al menos 1 uso real de `match/case`.
7. Debe existir al menos 1 transformación inmutable del estado.
8. Debe existir al menos 1 uso justificado de `map`, `filter` o `reduce`.
9. Debe haber al menos 1 fragmento donde expliquen por qué **no** conviene usar `lambda`.
10. Todo código asistido por IA debe poder explicarse técnicamente.

Entregables obligatorios

1. Aplicación funcional

Entrega del código fuente completo del proyecto.

2. Documento de análisis: `analisis-funcional.md`

Debe incluir obligatoriamente:

1. problema que resuelve la aplicación,
2. estructuras de datos utilizadas,
3. dónde se aplicó Unidad 1,
4. dónde se aplicó Unidad 2,
5. qué partes son puras y cuáles impuras,
6. qué efectos secundarios existen,
7. qué parte del estado se manejó de forma inmutable,
8. qué closure se implementó y para qué,
9. qué partes sugirió la IA,
10. qué corrigieron o rechazaron de la IA,
11. qué aprendieron técnicamente.

3. Bitácora de uso de IA: `bitacora-ia.md`

Debe incluir:

- prompts usados,
- fragmentos sugeridos por IA,

- qué se aceptó,
- qué se modificó,
- qué se rechazó,
- y por qué.

4. Video de explicación y archivo de enlace

Cada estudiante deber? grabar un **video de explicaci?n** del proyecto y subir el enlace en un archivo de texto llamado `enlace-video.txt`.

El video debe permitir evidenciar que ustedes comprenden técnicamente su solución. En ese video deben explicar, como mínimo:

- dónde está la función de orden superior;
- dónde está el `match/case` y por qué se usó;
- cuál es el closure real del proyecto;
- qué parte es pura y cuál es impura;
- dónde hay inmutabilidad;
- qué parte de la interfaz produce efectos secundarios;
- qué hizo la IA y qué corrigieron ustedes.

El archivo `enlace-video.txt` debe contener únicamente el enlace final y funcional del video entregado.

Criterios de evaluación sugeridos

Criterio	**Puntaje**
Interfaz y experiencia de usuario	4 puntos
Aplicación real de la Unidad 1	6 puntos
Aplicación real de la Unidad 2	6 puntos
Calidad del diseño del código	4 puntos
Documento de análisis	3 puntos
Uso crítico de IA	2 puntos
Total	**25 puntos**

Qué se penalizará con fuerza

Se penalizará con severidad si ocurre alguna de estas situaciones:

- toda la lógica está metida dentro de callbacks o botones;
- se usó IA, pero no pueden explicar el código;

- se usaron lambdas solo para “parecer funcionales”;
- se confunde inmutabilidad con copiar por copiar sin sentido;
- se declara que una función es pura cuando hace `print`, `input`, acceso a archivos o mutación externa;
- se usa `match` solo como adorno;
- el proyecto se ve bonito, pero conceptualmente está vacío.

Mensaje clave

> La interfaz puede ser asistida por IA, pero la comprensión técnica es responsabilidad del estudiante.
>
> No se evaluará solamente que el sistema funcione, sino la capacidad de demostrar, justificar y defender la aplicación real de los conceptos de las Unidades 1 y 2.
>
> La IA es una herramienta de apoyo. El criterio, el análisis y la comprensión siguen siendo humanos.

Sentido formativo de la actividad

Esta actividad busca que ustedes:

- diseñen algo real,
- usen herramientas modernas,
- reconozcan cuándo la IA ayuda y cuándo estorba,
- demuestren que entienden programación funcional más allá de copiar sintaxis.

El producto final no debe ser solo una app “bonita”, sino una solución que muestre **pensamiento funcional, criterio técnico y autonomía intelectual**.